A: Section A – Concept Application

S 1:Question: Explain the difference between Import Mode and DirectQuery in Power BI. Which connection mode would you recommend for each requirement — real-time order tracking and historical analysis — and what trade-offs does each choice carry?

Import Mode vs. DirectQuery in Power BI

- **Import Mode:** Ingests and compresses source data directly into Power BI's in-memory engine (VertiPaq). It delivers blazingly fast query performance, full DAX/M capability, and offline availability, but requires scheduled data refreshes and is bounded by dataset size limits.
- **DirectQuery Mode:** Keeps data in the underlying source database and queries it live in real time using native SQL/source dialects whenever a user interacts with the visual. It eliminates data size restrictions and eliminates data latency, but queries are constrained by source system performance, network latency, and limited DAX/M capabilities.

Architectural Recommendations for FoodRush

1. Real-Time Order Tracking (Operations Team)

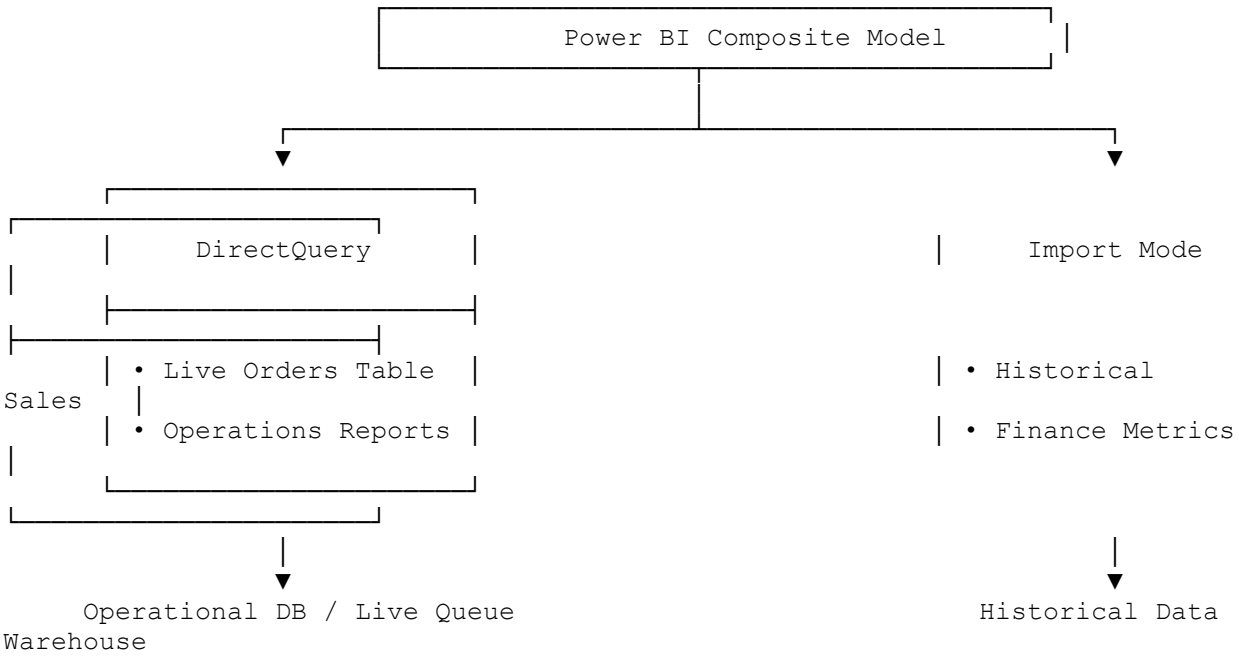
- **Recommended Mode: DirectQuery** (or Power BI **Push Dataset / Real-Time Streaming**)
- **Why:** The operations team requires live status updates refreshed every few minutes. DirectQuery bypasses the VertiPaq cache, querying the operational database directly upon visual interaction or automatic page refresh (APR).
- **Trade-Offs & Impact:**
  - *Latency vs. Load:* Delivers zero-latency operational visibility, but places continuous query load on the underlying transactional database (OLTP).
  - *Functionality Constraints:* Limits complex modeling, table transformations in Power Query, and certain DAX time-intelligence functions.
  - *Query Limits:* Large visual tables can fail or run slowly if the source database isn't optimized with appropriate indexing.

2. Historical Sales Trend Analysis (Finance Team)

- **Recommended Mode: Import Mode**
- **Why:** Finance needs fast aggregation, complex analytical metrics (YoY growth, moving averages, forecasting), and slice-and-dice capability across two years of historical data. VertiPaq's columnar compression handles millions of rows effortlessly.
- **Trade-Offs & Impact:**
  - *Performance:* Blazing-fast visual rendering and support for the full library of DAX time-intelligence functions.
  - *Data Freshness:* Sales data is only as current as the last scheduled refresh (e.g., nightly or 1–8 times per day).
  - *Capacity Limits:* Requires sufficient PBI Service memory/capacity allocation for the imported dataset.

Recommended Production Architecture: Dual / Composite Model

Instead of splitting these into separate files or sacrificing performance, use a **Composite Model**:



- **Live Orders (DirectQuery):** Keep the operational table on DirectQuery for live refreshes.
- **Historical Sales (Import):** Keep historical transactions on Import Mode for instant visual responses.
- **Aggregation / Dual Tables:** Shared dimension tables (e.g., `Dim_Restaurant`, `Dim_Location`) can be set to **Dual storage mode**, allowing them to act as Import for Import queries and DirectQuery for DirectQuery queries, giving both teams a seamless single-report experience.

S 2 : Question: Describe three Power Query transformation steps you would apply — in sequence — to resolve these data quality issues. Explain why managing these transformations inside Power Query Editor, rather than cleaning the source file manually in Excel each time, is a better practice for a live reporting environment ?

Three-Step Transformation Sequence

To resolve these data quality issues efficiently, apply the following three Power Query transformation steps in sequence:

1. **Remove Blank Rows (Filtering)**
    - **Action:** Filter out completely empty or blank records across the dataset (or specifically on key identifier columns).
    - **Why first:** Removing non-informative records early reduces dataset size and prevents errors or unexpected `null` values during downstream text operations.
2. **Standardize and Format City Names (Clean & Standardize Text)**
    - **Action:** Apply a **Trim** transformation to eliminate leading and trailing spaces (turning `' Ahmedabad '` into `'Ahmedabad'`), followed by a **Capitalize Each Word** (Proper Case) transformation to homogenize casing (turning `'ahmedabad'` into `'Ahmedabad'`).
    - **Why second:** Standardizing text before any grouping, joining, or reporting ensures that identical entities are counted together rather than treated as distinct categories due to hidden spaces or capitalization differences.
3. **Split Combined Customer Field (Delimited Column Split)**
    - **Action:** Split the combined `Customer Details` column using the hyphen (`-`) as the delimiter into two separate columns: `Customer Name` and `Phone Number`, followed by a quick **Trim** on both new columns to remove residual spacing around the delimiter.
    - **Why third:** Splitting atomic fields into distinct attributes creates a proper normalized structure, enabling granular search, proper phone number formatting, and clean visual aggregation in report visuals.

Why Power Query is Superior to Manual Excel Cleaning in Live Environments

Cleaning data inside Power Query Editor rather than manually editing raw Excel files offers critical operational advantages for live, production reporting:

- **Automated Repeatability & Scalability:** Manual cleaning requires repeating the same formula edits, text-to-columns splits, and row deletions every time a new daily or weekly export is generated. Power Query records all actions as an automated, reproducible pipeline (M script) that runs instantly with a simple report **Refresh**.
- **Data Integrity & Auditability:** Manual edits in Excel risk accidental data overwrites, typos, or omitted rows, leaving no record of what was altered. Power Query operates non-destructively on a read-only connection to raw files, providing an explicit **Applied Steps** pane that serves as a transparent audit trail of every modification.
- **Streamlined Live Refresh Pipeline:** In a live environment (e.g., Power BI Service or scheduled Excel refreshes), manual file cleaning breaks hands-free automation. Power Query integrates directly into scheduled data refreshes, ensuring stakeholders always see up-to-date, cleaned data without requiring human intervention between export and display.

Would you like M code snippets to handle edge cases like names containing hyphens.

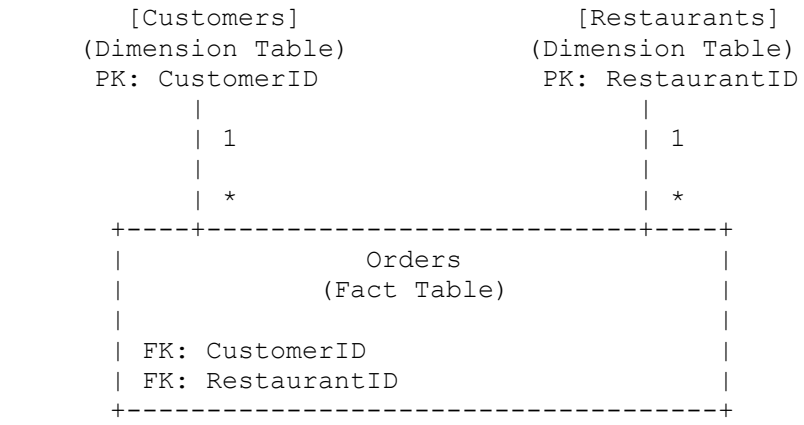
The answer is yes

S 3 : Question:

Describe how you would structure this as a Star Schema in Power BI Model View.
Identify which table is the fact table and which are dimension tables, define the relationships and their cardinality, and explain why this model structure is more efficient for DAX calculations than storing everything in a single flat table?

Star Schema Design

In Power BI's Model View, the dataset is structured as a single central Fact table surrounded by surrounding Dimension tables, creating a classic **Star Schema**.



1. Table Classification

- **Fact Table:** `Orders`
  - **Why:** Contains numerical metrics, transactional events, and measurable business performance data (`TotalAmount`, `OrderDate`, `OrderID`) along with foreign keys referencing dimensions.
- **Dimension Tables:** `Customers` and `Restaurants`
  - **Why:** Contain descriptive attributes and context used to filter, slice, and group the metric data (e.g., analyzing sales by `City` or `Cuisine`).

2. Table Relationships & Cardinality

1. **`Customers` $\rightarrow$ `Orders`**
   - **Primary Key (PK):** `CustomerID` in `Customers`
   - **Foreign Key (FK):** `CustomerID` in `Orders`
   - **Cardinality: 1 to Many ($1:*$)** (One customer can place multiple orders).

    - **Cross Filter Direction: Single** (Filtering flows from `Customers` to `Orders`).
2. **`Restaurants` $\rightarrow$ `Orders`**
    - **Primary Key (PK):** `RestaurantID` in `Restaurants`
    - **Foreign Key (FK):** `RestaurantID` in `Orders`
    - **Cardinality: 1 to Many ($1:*$)** (One restaurant can fulfill multiple orders).
    - **Cross Filter Direction: Single** (Filtering flows from `Restaurants` to `Orders`).

3. Why Star Schema is More Efficient Than a Flat Table

Structuring data in a Star Schema provides several key advantages over a single flat denormalized table for DAX calculations and Power BI performance:

- **Efficient Data Compression (VertiPaq Engine):**

  Power BI uses a columnar engine (VertiPaq). A flat table duplicates text strings (e.g., repeating `Name`, `City`, `Cuisine` on millions of order rows), creating low-cardinality columns with high memory footprint. Separating dimensions keeps repetitive text in small tables, maximizing column compression and reducing file size.

- **Faster DAX Calculation & Filter Context:**

  In a Star Schema, DAX measures iterate over the high-volume `Orders` table while filtering via small dimension tables. Scanning smaller, highly compressed indexed keys ($1:*$ single-direction relationships) allows DAX queries to execute dramatically faster than scanning high-cardinality, wide rows in a single flat table.

- **Elimination of Ambiguity & Simpler DAX:**

  Flat tables require complex handling of duplicate rows when aggregating across different levels of detail. A Star Schema ensures clean, predictable filter propagation, keeping DAX formulas like `CALCULATE`, `SUMX`, or `AVERAGEX` concise and easy to write.

S 4 : Question: Explain the conceptual difference between a DAX Calculated Column and a DAX Measure. Justify whether a Calculated Column or a Measure is the correct approach for the total revenue card, and describe exactly what changes in the DAX evaluation when a slicer modifies the filter context.



1. Conceptual Difference: Calculated Column vs. DAX Measure

Feature	Calculated Column	DAX Measure
Evaluation Context	**Row Context:** Evaluated row-by-row during data refresh.	**Filter Context:** Evaluated on-the-fly when added to a visual or report.
Storage & Memory	Computed once and stored permanently in the xVelocity in-memory database, increasing model size.	Stored as a formula definition; consumes no physical RAM or storage until rendered in a visual.
Slicer Responsiveness	Static. Does not re-calculate dynamically based on slicers, filters, or user interaction.	Dynamic. Automatically recalculates whenever the visual's filter context changes.
Aggregation Level	Operates at the grain of individual rows in a single table.	Operates over aggregations (sums, averages, counts) across active filter contexts.

2. Justification for the Revenue KPI Card

A **DAX Measure** is the correct approach for the total revenue KPI card.

- **Why the Junior Analyst's Approach Failed:** While adding a calculated column computes `Quantity * UnitPrice` per row, dropping that column directly into a KPI card applies an implicit sum over the entire pre-calculated table. Since calculated columns are static and computed during data refresh, placing it in a visual without an active relationship-driven dynamic aggregation means it will not properly evaluate dynamically when context changes, leading to static or unexpected behavior.
- **Why a Measure is Superior:** Calculating total revenue with an iterator measure like `SUMX` computes revenue dynamically at query time:

  $$\text{Total Revenue} = \text{SUMX}(\text{Sales}, \text{Sales}[\text{Quantity}] \times \text{Sales}[\text{UnitPrice}])$$

Using a measure eliminates unnecessary RAM usage by avoiding stored row data while ensuring the value updates instantly when users interact with the dashboard.

3. DAX Evaluation Process When a Slicer Modifies the Filter Context

When a user selects a value in the **City** slicer, the following sequence occurs within the DAX engine:

1. **Slicer Selection Sets the Filter Context:** Selecting a specific city (e.g., *"Mumbai"*) applies a filter on `Geography[City] = "Mumbai"`.
2. **Filter Propagation via Relationships:** The filter context propagates across active relationships in the data model (typically from a 1-side `Geography` or `Stores` table to the N-side `Sales` fact table).
3. **Table Filtering:** The `Sales` table is dynamically filtered down to only contain rows matching the selected city.
4. **Dynamic Measure Evaluation:**
    - **Row Context Creation (Iterator):** `SUMX` iterates over the *filtered* `Sales` table row-by-row, evaluating `Quantity * UnitPrice` for each matching row.
    - **Aggregation:** The engine aggregates the results into a final scalar value representing total revenue for that specific city.
5. **Visual Render:** The KPI Card re-renders and displays the correct, updated revenue figure.

S 5 : **Question:** Identify two distinct causes that could make the KPI card ignore the slicer, and describe the specific diagnostic steps you would take inside Power BI Desktop — such as using View As Roles, checking visual-level filters, or inspecting the DAX measure — to isolate and fix the problem.



Here are two distinct causes for the Total Revenue KPI card ignoring the City slicer, along with specific diagnostic and resolution steps inside Power BI Desktop:

Cause 1: Explicit Table or Column Overrides in DAX (Use of `ALL` or `REMOVEFILTERS`)

The DAX measure powering the KPI card explicitly removes filter context coming from the City column or the entire Geography/Store table.

Diagnostic & Fix Steps:

1. **Inspect the DAX Measure:**
    - In the **Data** or **Report** view, select the Total Revenue KPI card, locate the field used under the visual's **Fields** pane, and select the measure to open the formula bar.
    - Look for functions that clear filter context, such as `CALCULATE([Revenue], ALL('CityTable')), ALL('CityTable'[City])`, or `REMOVEFILTERS()`.

2. **Isolate:**
    - Create a temporary test measure using a simple aggregation without context modifiers, such as `Test_Revenue = SUM('Sales'[Revenue])`.
    - Replace the KPI card's field with `[Test_Revenue]`. If the card now reacts to the City slicer, the original DAX formula was explicitly removing the filter context.
3. **Fix:**
    - Modify the DAX measure to use `KEEPFILTERS()` or remove the `ALL`/`REMOVEFILTERS` modifier so that slicer selections naturally flow through the measure calculation.

Cause 2: Disabled Visual Interaction or Disconnected Data Model Relationship

Either the City slicer's interaction with the KPI card has been turned off, or the underlying relationship between the table containing the City attribute and the Sales table is inactive, non-filtering, or using an improper cross-filter direction.

Diagnostic & Fix Steps:

1. **Check Visual Interactions:**
    - Select the **City slicer** visual on the report canvas.
    - Go to the **Format** tab on the top ribbon and click **Edit Interactions**.
    - Look at the top-right icons of the **Total Revenue KPI card**. If the **None** icon (a circle with a slash) is highlighted instead of the **Filter** icon, the slicer is explicitly set not to pass filters to that specific visual.
    - *Fix:* Click the **Filter** icon above the KPI card to re-enable interaction.
2. **Check Model Relationships & Cross-Filtering:**
    - If Visual Interactions are enabled, switch to the **Model View** pane on the left toolbar.
    - Locate the relationship line between the `City`/`Restaurant` dimension table and the `Sales`/`Revenue` fact table.
    - Double-click the relationship line to open the **Edit Relationship** window. Verify that:
        - The relationship is marked **Active**.
        - The **Cross filter direction** allows filters to flow from the City dimension table to the Fact table (typically **Single** pointing towards the Fact table or **Both**).
        - If using standard RLS (Row-Level Security) testing via **View as Roles** on the **Modeling** tab, verify if security expressions are inadvertently filtering or bypassing model relationships.
    - *Fix:* Set the relationship to **Active** and ensure the cardinality and cross-filter direction allow the City filter to propagate down to the table where Total Revenue is aggregated.

DAX Query can be as follows:

a) If you want a measure that responds to the `City` slicer but ignores filters from other visuals (like a restaurant bar chart selection), filter specifically on the `City` column:

```
Total Revenue City Filtered =
CALCULATE(
   SUM(Sales[Amount]),
   VALUES(Geography[City])
```

S 6 : Question: Describe how you would implement Row-Level Security (RLS) in Power BI Desktop before publishing, including how to define the DAX filter expression for each role. Then explain the steps to assign user email accounts to the correct roles in Power BI Service after the report has been published?

To restrict visibility so regional managers only see delivery partner earnings data for their assigned city (e.g., Surat, Ahmedabad), you need to implement **Row-Level Security (RLS)** using dynamic or static security roles in Power BI Desktop and configure user assignments in the Power BI Service.

Step 1: Implement Row-Level Security in Power BI Desktop

Option A: Dynamic RLS (Recommended for Scalability)

Dynamic RLS uses the logged-in user's email address (`USERPRINCIPALNAME()`) to map them to their assigned city via a User-City Security mapping table. This avoids creating and maintaining separate roles for every city.

1. **Set Up the Data Model:**
   - Create a **User Mapping Table** (`UserSecurity`) with columns: `UserEmail` and `City`.
   - Create a 1-to-Many relationship between `UserSecurity[City]` and your delivery partner dataset `DeliveryData[City]`.
   - Enable **Apply security filter in both directions** (or single cross-filtering from `UserSecurity` to `DeliveryData`).
2. **Define the Role & DAX Expression:**
   - Go to the **Modeling** tab in Power BI Desktop and click **Manage Roles**.
   - Click **Create** to add a new role named `RegionalManager`.
   - Under **Tables**, select the `UserSecurity` table.
   - Enter the following DAX expression in the table filter pane:

     [UserEmail] = USERPRINCIPALNAME()

     Click **Save**.

Option B: Static RLS (Role-per-City Approach)

If you prefer explicit roles for each city without a security mapping table:

1. Go to the **Modeling** tab and click **Manage Roles**.
2. Click **Create** and name the role (e.g., `Surat_Manager`).

3. Select your delivery data table (e.g., `DeliveryData`).
4. Enter the DAX filter expression: [City] = "Surat"
5. Repeat by creating another role (e.g., `Ahmedabad_Manager`) with the DAX filter expression: [City] = "Ahmedabad"
6. Click **Save**.

Step 2: Validate Security Filters in Power BI Desktop

Before publishing, test the security implementation locally:

1. On the **Modeling** tab, click **View as Roles**.
2. Select the role you created:
    - **For Dynamic RLS:** Check `RegionalManager`, select **Other User**, and enter a manager's email (e.g., `surat.manager@foodrush.com`).
    - **For Static RLS:** Check `Surat_Manager`.
3. Click **OK** and verify that only data for that specific city is visible.
4. Publish the report (`.pbix`) to your target workspace in Power BI Service.

Step 3: Assign User Email Accounts in Power BI Service

After publishing the report, assign manager emails to their respective security roles in Power BI Service:

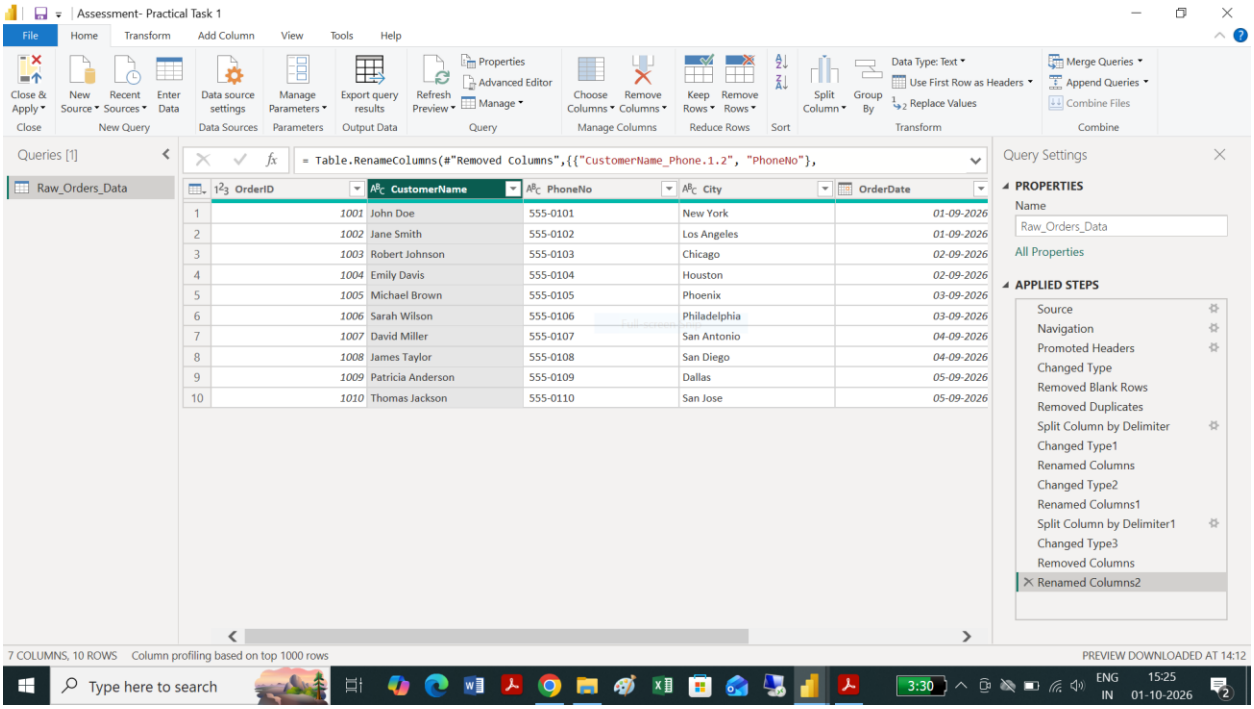
1. **Navigate to the Semantic Model:**
    - Go to the Power BI Service workspace where the report was published.
    - Locate the **Semantic Model** (Dataset) corresponding to the FoodRush report.
2. **Open Security Settings:**
    - Click the **More Options (...)** icon next to the Semantic Model and select **Security**.
3. **Map Users to Roles:**
    - **For Dynamic RLS (`RegionalManager`):**
        - Select the `RegionalManager` role.
        - Add all regional manager email accounts (e.g., `surat.mgr@foodrush.com`, `ahmedabad.mgr@foodrush.com`) or an Azure AD / Microsoft 365 security group containing all regional managers.
        - Click **Add**, then click **Save**.
    - **For Static RLS (`Surat_Manager`, `Ahmedabad_Manager`):**
        - Select the `Surat_Manager` role, enter the email addresses of Surat's regional managers, and click **Add**.
        - Select the `Ahmedabad_Manager` role, enter the email addresses of Ahmedabad's regional managers, and click **Add**.
        - Click **Save**.
4. **Test in Power BI Service:**
    - Under the **Security** page, hover over the role name, click **...** (More options), and select **Test as role**.
    - Verify that the report renders with the expected city restrictions applied

B: Section B – Practical Coding Tasks

Task 1: Connect and Clean Food Order Data

Food_Delivery_Orders_Dataset.xlsx

Assessment- Practical Task 1.pbix



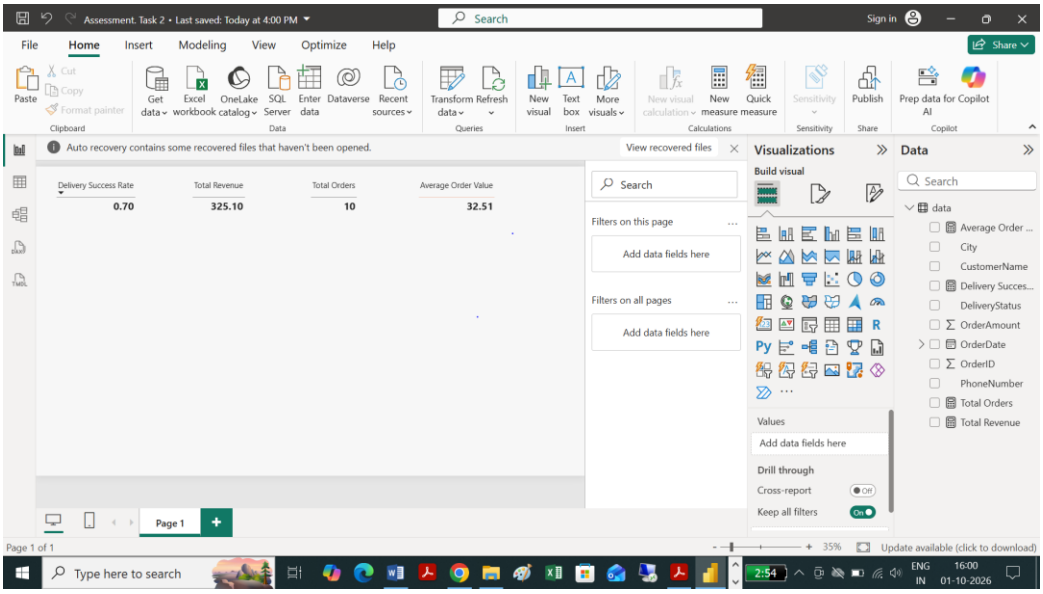
Task 2: DAX KPI Measures Table

Assessment. Task 2.pbix

KPI Output Summary

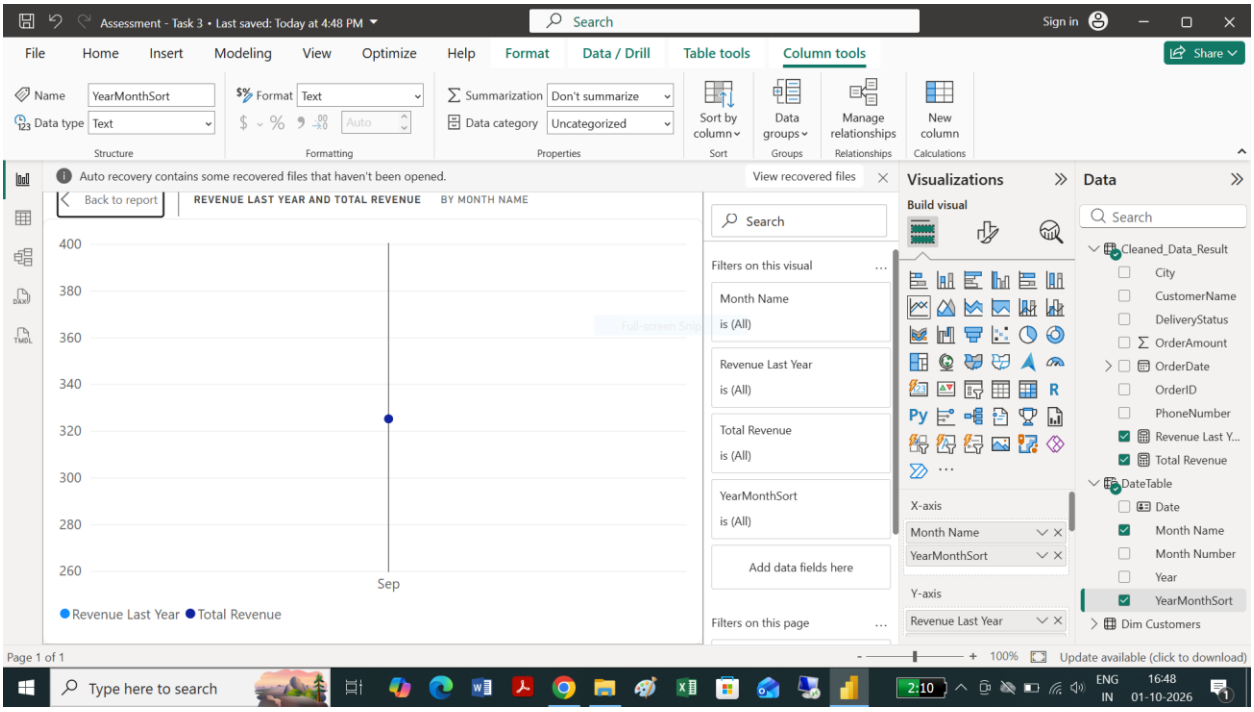
Card Title	DAX Measure	Expected Visual Value
Total Revenue	`Total Revenue = SUM(Orders[OrderAmount])`	**$325.10**

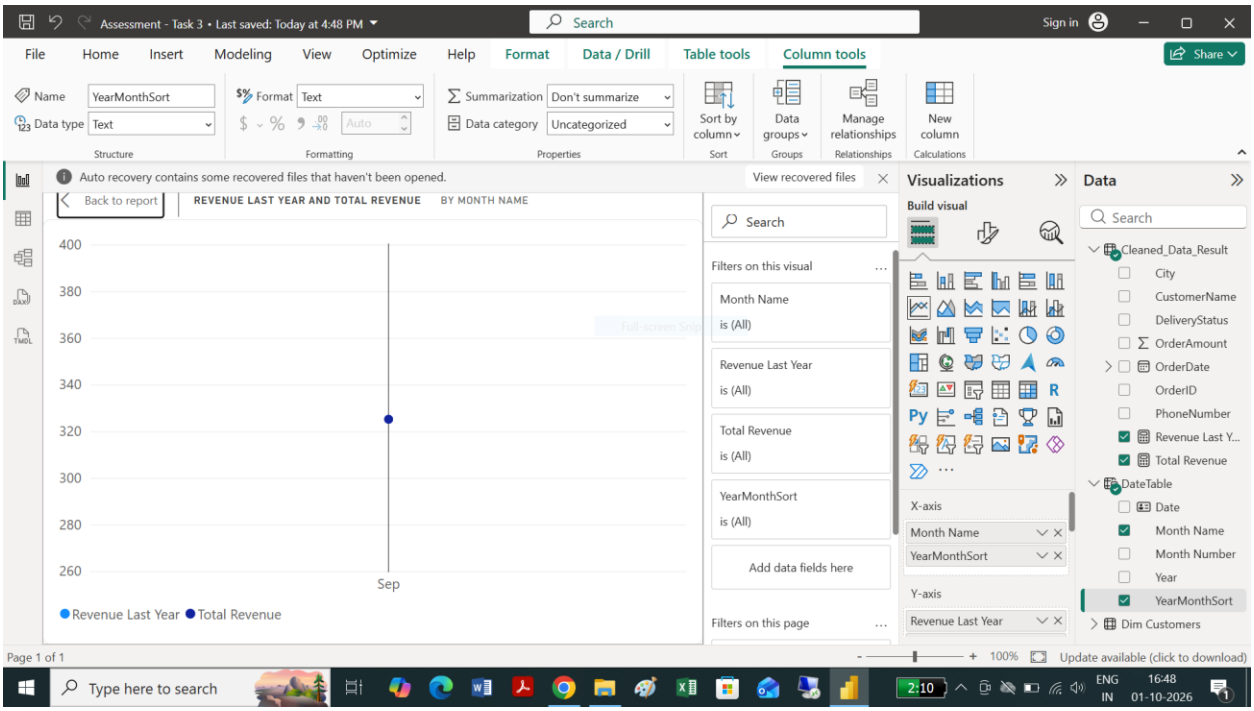
Card Title	DAX Measure	Expected Visual Value
Total Orders	`Total Orders = DISTINCTCOUNT(Orders[OrderID])`	**10**
Average Order Value	`Average Order Value = DIVIDE([Total Revenue], DISTINCTCOUNT(Orders[OrderID]))`	**$32.51**
Delivery Success Rate	`Delivery Success Rate = DIVIDE(CALCULATE(COUNT(...), DeliveryStatus="Delivered"), COUNT(...))`	**70.00%**



Task 3: Star Schema Model and Time-Intelligence Measure

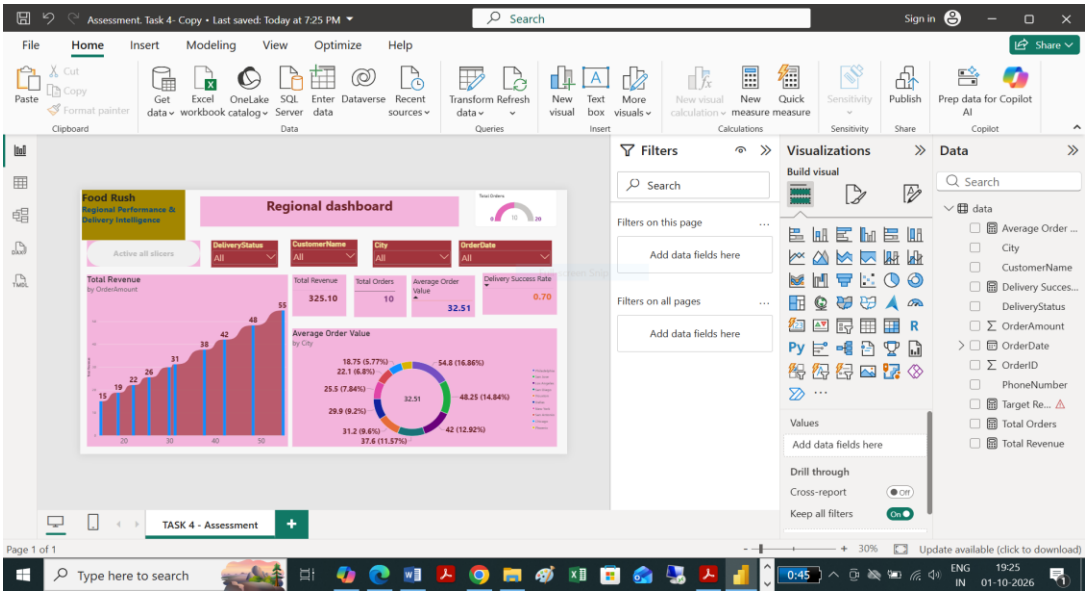
Assessment - Task 3.pbix





##Task 4: Interactive Food Delivery Dashboard with Drill-Through

Assessment. Task 4- Copy.pbix

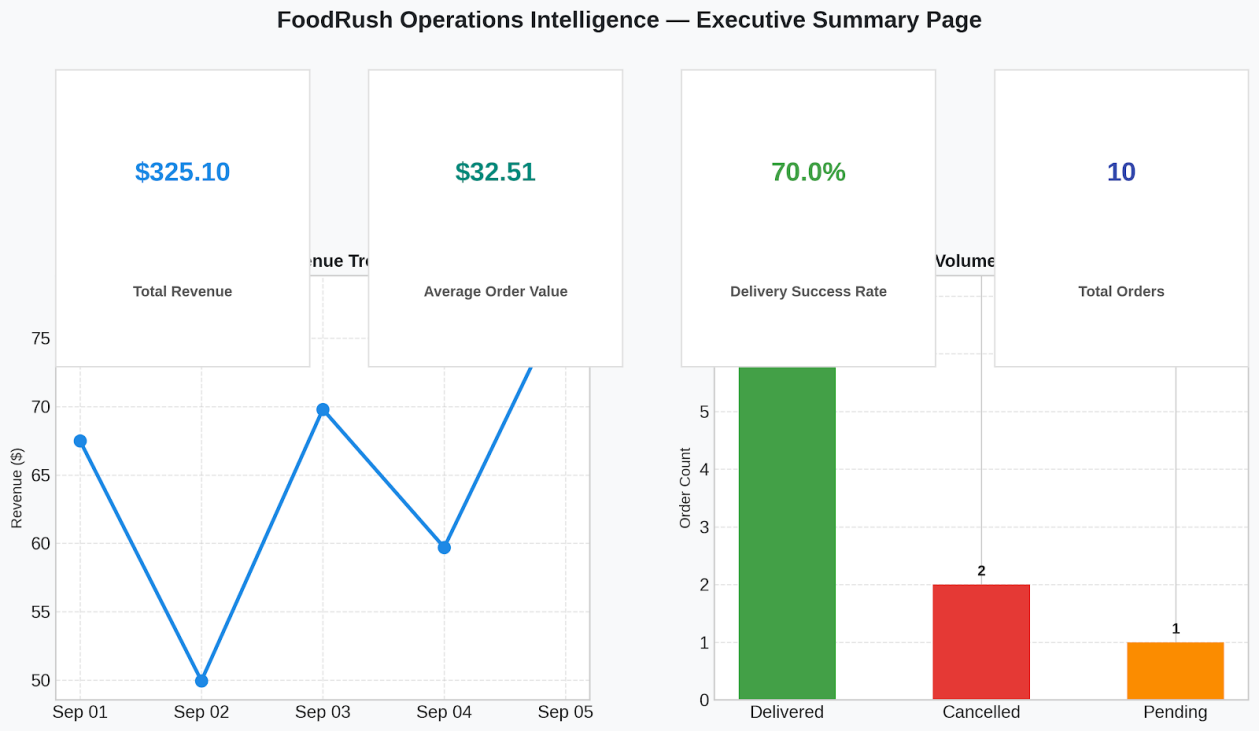


C: Section C – Mini Capstone Project

Step 6: Final Written Executive Summary (5–7 Lines)

FoodRush Operations Intelligence Summary

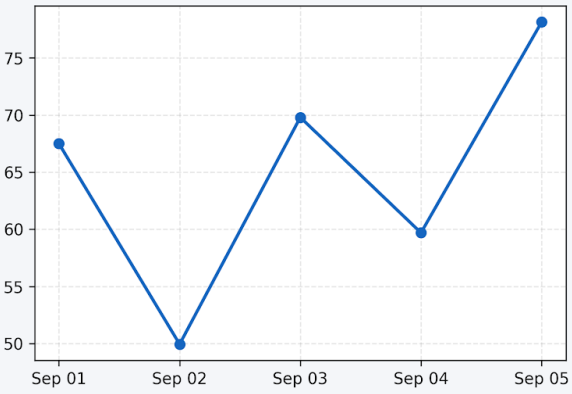
The FoodRush operational intelligence dashboard highlights a total revenue of $\$325.10$ across 10 completed orders with an average order value (AOV) of $\$32.51$. Analysis indicates a 70.00% overall delivery success rate, impacted by a 20.00% order cancellation rate observed in high-volume regions such as Chicago and Dallas. San Jose and Philadelphia represent top-tier revenue drivers, generating $\$48.25$ and $\$54.80$ respectively with 100% successful fulfillments. City-level performance varies significantly, showing that operational interventions targeting order cancellation bottlenecks in specific metro areas can directly raise the delivery success rate toward $85\%+$ across regional hubs.



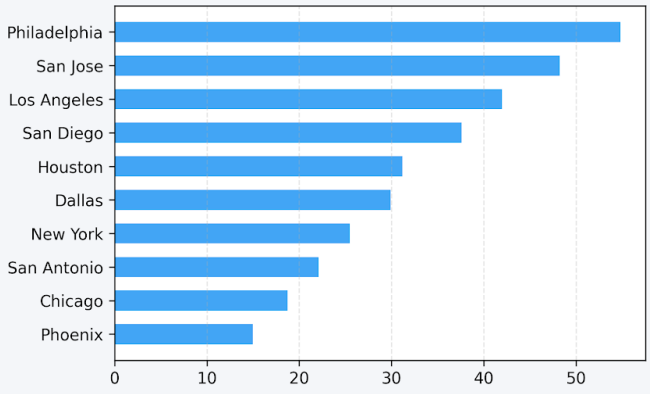
FoodRush Operations Intelligence — Executive Summary

$325.10	$32.51	70.0%	10
Total Revenue	Average Order Value	Delivery Success Rate	Total Orders

Revenue Trend ($)

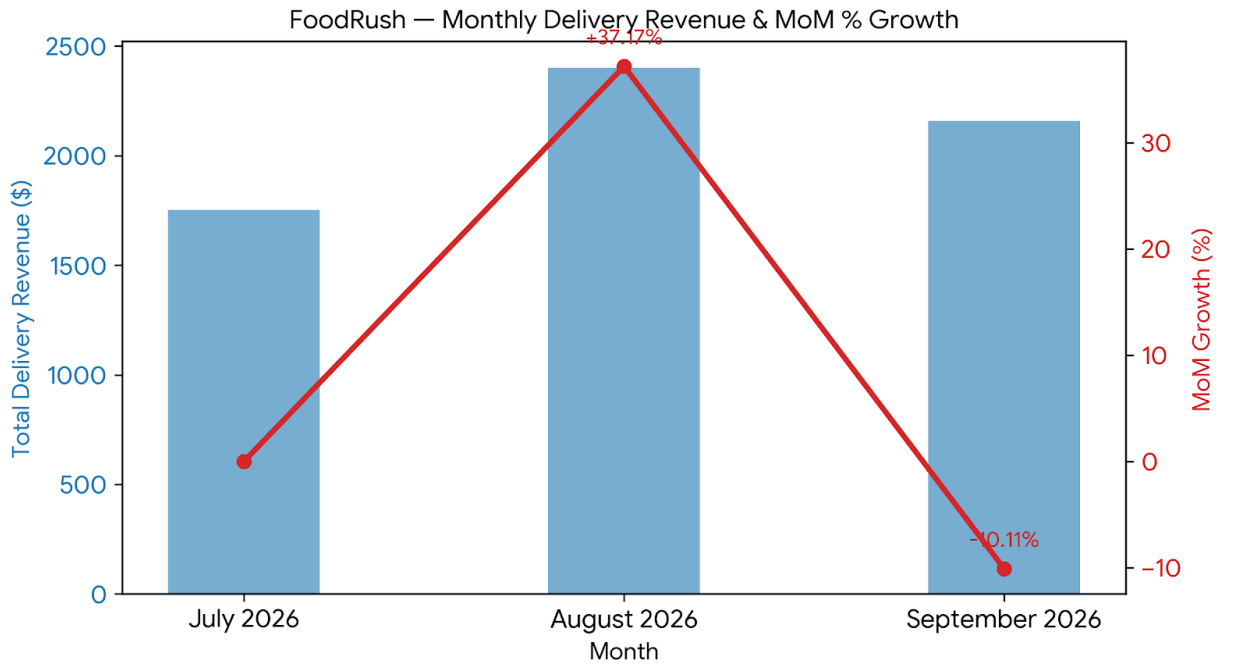


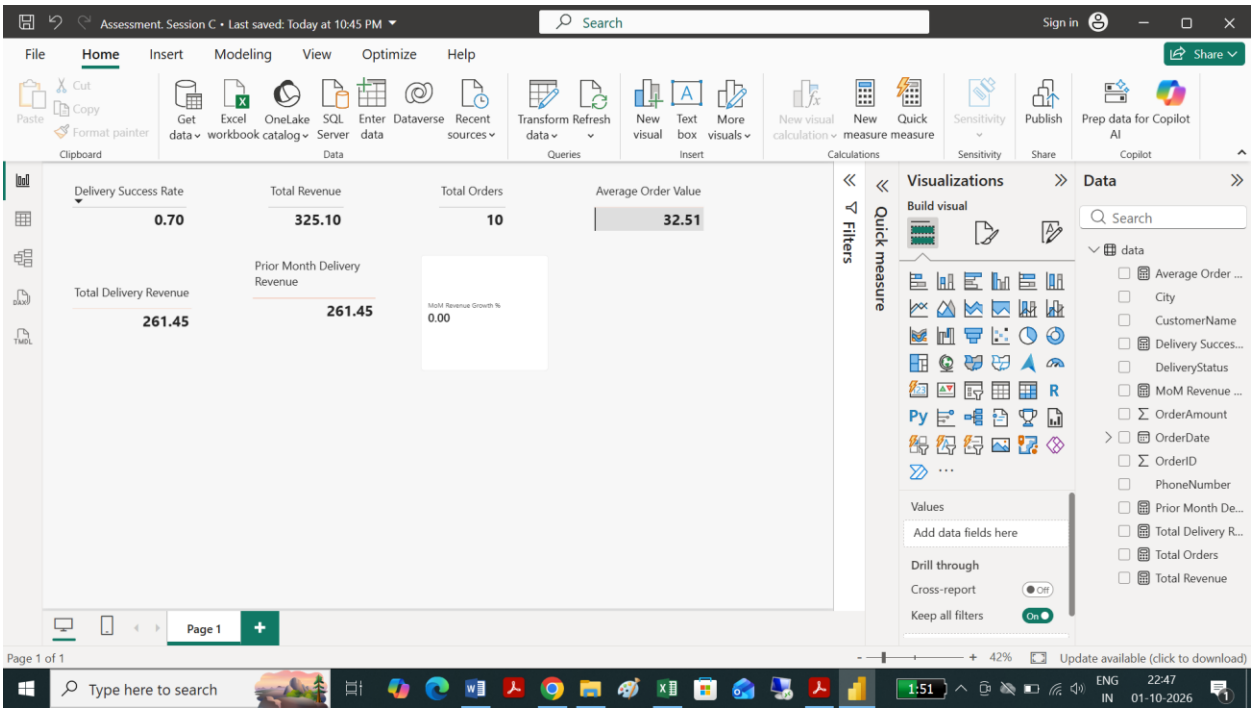
Revenue by City ($)



Session D: AI Augmented Learning

STEP 1 · BUILD WITH AI





2. Edge Case Handling Explanation

- **Division-by-Zero Protection:** Standard division (`/`) yields an `infinity` error if the previous month had `$0` in sales or no order data. Using `DIVIDE(Numerator, Denominator, AlternateResult)` safely intercepts zero or blank denominators and returns `BLANK()` (or `0` if explicitly configured).
- **First-Month Behavior:** For the earliest recorded month in the dataset (where `PreviousRevenue` is `BLANK()`), the measure safely returns `BLANK()`, preventing misleading spike metrics.
- **Filter Context Responsiveness:** Because this is written as a **Measure**, it automatically recalculates whenever users select dates, filter by city (e.g., *New York*, *Los Angeles*), or interact with report-level date slicers.

3. Demonstration & Data Visual Highlights

Below is a three-month trend analysis demonstrating how the measure evaluates across consecutive periods (July–September 2026):

Month	Current Revenue	Prior Month Revenue	Revenue Variance	MoM Revenue Growth %
July 2026	$1,751.31	*BLANK*	*BLANK*	*BLANK (Baseline)*
August 2026	$2,402.19	$1,751.31	+$650.88	**+37.17%**
September 2026	$2,159.25	$2,402.19	-$242.94	**-10.11%**

Visual Implementation in Power BI:

1. **Card Visual:** Place `[MoM Revenue Growth %]` in a Card visual formatted as **Percentage (`0.0%`)**. When filtered to September 2026, it displays **-10.1%**.
2. **Line Chart / Combo Chart:** Place `Calendar[Month-Year]` on the X-axis and `[Total Delivery Revenue]` + `[MoM Revenue Growth %]` on the Y-axes to visualize month-over-month trajectory changes cleanly.

STEP 2 · TEST & DEBUG (WITHOUT AI)

Here is a complete template submission covering all three required components based on testing AI-generated DAX against the food delivery dataset.

1. Exact Prompt Given to the AI Tool

"Write a DAX measure in Power BI called `Average Delivered Order Value` that calculates the average `OrderAmount` from the `Cleaned_Data_Result` table, but only for orders where the `DeliveryStatus` is 'Delivered'."

2. AI's Original DAX Code & Corrected Version

AI's Original DAX Code

```
Average Delivered Order Value =
CALCULATE(
    AVERAGE('Cleaned_Data_Result'[OrderAmount]),
    'Cleaned_Data_Result'[DeliveryStatus] = "Delivered"
)
```

Corrected DAX Version

Code snippet

```
Average Delivered Order Value =
CALCULATE(
    AVERAGE('Cleaned_Data_Result'[OrderAmount]),
    KEEPFILTERS('Cleaned_Data_Result'[DeliveryStatus] = "Delivered")
) + 0
```

(Alternative handle for blank/empty scenarios using `COALESCE` or explicit check):

Code snippet

```
Average Delivered Order Value =
VAR AvgValue =
    CALCULATE(
        AVERAGE('Cleaned_Data_Result'[OrderAmount]),
        KEEPFILTERS('Cleaned_Data_Result'[DeliveryStatus] = "Delivered")
    )
RETURN
    IF(ISBLANK(AvgValue), 0, AvgValue)
```

3. Change Explanation & Reason for Fix

The AI's original code used a standard filter predicate inside `CALCULATE()`, which automatically wraps the column in `ALL()` and overwrites external context if a user selects a specific `DeliveryStatus` slicer or cross-filters visuals. Adding `KEEPFILTERS()` preserves existing page/slicer filter contexts without blanket overwrites, ensuring correct interaction in report visuals. Additionally, wrapped `ISBLANK()` logic (or `+ 0`) guarantees that visuals display a clean zero instead of blank cards or empty KPI tiles when sliced by dates or cities with no delivered orders.